

FLAPPY BIRD GAME

PROJECT REPORT

A JavaFX Game Development Project

Object-Oriented Programming (OOP)

GROUP MEMBERS

- | | |
|----------------------|--------------|
| 1. Muhammad Abdullah | SP25-BSE-067 |
| 2. Abul Basit Shahid | SP25-BSE-001 |
| 3. Aizaz Noor | SP25-BSE-013 |

Submitted to: **Mr. Abdul Karim Shahid**

Course: **Object Oriented Programming**

Date: **12/14/2025**

TABLE OF CONTENTS

1. Introduction
2. Objectives
3. UML Diagram
4. Modular Programming (Classes and OOP)
5. Game Logic (Gravity, Jumping, Collision)

6. User Interface
7. Features
8. Team Work
9. Conclusion

1. Introduction

For our semester project, we decided to build a clone of the famous game "**Flappy Bird**". We built this game from scratch using the **Java** programming language. We used **JavaFX** to draw the graphics on the screen.

The main reason we chose this project was to practice the "Object-Oriented Programming" (OOP) skills we learned in class. We wanted to see how concepts like classes, objects, and inheritance work in a real-world software project.

2. Objectives

- To implement a fully functional Flappy Bird game using Java and JavaFX.
- To demonstrate Object-Oriented Programming principles through modular design.
- To implement game physics including gravity and collision detection.
- To create an interactive user interface with keyboard and mouse controls.
- To integrate multimedia features like background music and sound effects.

3. UML Diagram

Class: GameEngine

Description: The central controller that manages the game loop, object updates, and rendering.

Attributes (Fields)

- <code>AnimationTimer gameLoop</code>	The main rendering loop running at 60 FPS.
- <code>Bird bird</code>	The player object instance.
- <code>List<Pipe> pipes</code>	A collection of all active obstacles.
- <code>GameState gameState</code>	Tracks current state (PLAYING, MENU, GAME_OVER).
Methods	
+ <code>update(long now)</code>	Updates physics and game logic for the current frame.
+ <code>render()</code>	Draws the background, bird, pipes, and UI to the canvas.
- <code>gameOver()</code>	Stops the game loop and triggers the Game Over screen.

Class: Bird

Description: Represents the player character with physics properties.

Attributes	
- <code>double y</code>	Vertical position (Height).
- <code>double velocity</code>	Current vertical speed (falling or jumping).

- double GRAVITY	Constant force (0.5) applied every frame.
Methods	
+ jump()	Sets velocity to negative value to move upwards.
+ update()	Applies gravity to velocity and updates Y position.
+ render(GraphicsContext gc)	Draws the yellow bird circle with rotation.

Class: Pipe

Description: Represents a single obstacle column (Top and Bottom pipe).

Attributes	
- double x	Horizontal position.
- double gapY	The vertical center of the safe gap.
- boolean scored	True if the bird has successfully passed this pipe.
Methods	
+ update()	Moves the pipe to the left by SPEED pixels.

+ <code>collidesWith(Bird b)</code>	Returns TRUE if bird intersects with pipe rectangle.
-------------------------------------	--

Class: SoundManager

Description: Handles loading and playback of audio assets.

Attributes	
- <code>MediaPlayer backgroundMusic</code>	Loops the background theme.
- <code>AudioClip crashSound</code>	Plays when collision occurs.
Methods	
+ <code>playJump()</code>	Triggers wing-flap sound effect.
+ <code>dispose()</code>	Releases audio resources when app closes.

4. Modular Programming

Our teacher asked us to use "Modular Programming." This simply means breaking a big problem into small, easy pieces.

Instead of writing 1000 lines of code in one file, we created separate files (Classes) for each part of the game. This is good because:

- It makes the code easier to read.
- It allows us to fix one part without breaking another.
- It lets us reuse code.

Our Main Classes

- **Bird.java:** This file ONLY handles the bird. It knows the bird's position and speed. It doesn't know about pipes or scores.
- **Pipe.java:** This file ONLY handles the obstacles. It knows where the pipes are and how fast they move.
- **SoundManager.java:** This file ONLY handles audio. It loads the music files and plays them when needed.
- **GameEngine.java:** This is the "Brain" setup. It connects the Bird, Pipes, and Sounds together to make the game work.

OOP Principles Applied

A) Encapsulation

Encapsulation means hiding internal data and providing controlled access through methods. Each class uses `private` fields with `public` getter methods.

```
public class Bird {
    private double x, y;           // Private fields (hidden)
    private double velocity;

    public double getX() { return x; } // Public getter (access)
    public double getY() { return y; }
}
```

B) Composition

Composition is when one class contains objects of other classes. GameEngine uses composition to manage all game components.

```
public class GameEngine {
    private Bird bird;           // Contains Bird object
    private List<Pipe> pipes;    // Contains Pipe objects
    private SoundManager soundManager; // Contains SoundManager
    private SettingsMenu settingsMenu; // Contains SettingsMenu
}
```

C) Single Responsibility Principle

Each class has one specific job:

Class	Responsibility
FlappyBirdGame	Application startup and window creation
GameEngine	Game loop, input handling, state management
Bird	Player physics and rendering
Pipe	Obstacle behavior and collision detection
SoundManager	Audio loading and playback
SettingsMenu	Settings UI overlay

Benefits of Modular Design

- **Maintainability:** Easy to find and fix bugs in specific classes
- **Reusability:** Classes can be reused in other projects
- **Testability:** Each class can be tested independently
- **Readability:** Code is organized and easy to understand
- **Scalability:** Easy to add new features without breaking existing code

5. Game Logic (Gravity, Jumping, Collision)

This section explains the core game mechanics: how the bird falls, jumps, and how collisions are detected.

The Game Loop

The game runs on a continuous loop that executes **60 times per second** using JavaFX's `AnimationTimer`:

```
gameLoop = new AnimationTimer() {
    @Override
    public void handle(long now) {
        update(now);    // Step 1: Update game logic
        render();        // Step 2: Draw everything
    }
};
gameLoop.start();
```

Gravity Physics

Gravity makes the bird fall down continuously. This simulates real-world physics where objects accelerate as they fall.

The Physics Formula:

`velocity += GRAVITY;` → Add 0.5 to velocity each frame
`y += velocity;` → Move bird down by velocity amount

Step-by-step example (bird starts at `y=300` with `velocity=0`):

Frame	Velocity Calculation	New Velocity	Y Position
1	0 + 0.5	0.5	300.5
2	0.5 + 0.5	1.0	301.5

3	$1.0 + 0.5$	1.5	303.0
4	$1.5 + 0.5$	2.0	305.0
5	$2.0 + 0.5$	2.5	307.5

Notice how velocity increases each frame (acceleration) = realistic falling motion!

Jumping Mechanics

When the player presses SPACE or clicks, the bird jumps upward:

```
public void jump() {
    velocity = JUMP_STRENGTH; // Set velocity to -10 (upward)
}
```

Why negative? In screen coordinates: Positive Y = Down, Negative Y = Up. So `velocity = -10` means "move 10 units upward".

Collision Detection

The algorithm checks two conditions:

Step 1: Is bird horizontally aligned with pipe?

```
if (birdX + radius > pipeX && birdX - radius < pipeX + WIDTH)
```

Step 2: Is bird outside the safe gap?

```
double topPipeBottom = gapY - gapSize / 2;
double bottomPipeTop = gapY + gapSize / 2;

if (birdY - radius < topPipeBottom || birdY + radius > bottomPipeTop) {
    return true; // COLLISION!
}
```

Scoring System

```
if (pipe.isPassed(bird.getX())) {  
    pipe.setScored();    // Mark pipe as scored  
    score++;             // Increase score  
    if (score > highScore)  
        highScore = score; // Update high score  
}
```

The Loop (Heartbeat of the Game)

Video games are like flipbooks. They show a picture, move things slightly, and show a new picture. Our game does this 60 times every second.

1. UPDATE: We calculate new positions for the bird and pipes. 2. CHECK: We check if the bird crashed. 3. DRAW: We clear the screen and draw everything in the new spots. 4. REPEAT.

Gravity Logic

We don't just move the bird down. We perform a calculation:

- **Velocity (Speed):** Starts at 0.
- **Gravity:** A constant force (0.5).

Every frame, we say: $Speed = Speed + Gravity$.
So, at first, the bird falls slowly. Then it falls faster. And faster. This creates realistic physics.

The Scoring System

How do we know when to give a point?

Every pipe has a variable called `scored` which starts as `false`. As the game runs, we check: *"Is the bird's X position greater than the pipe's X position?"*

If the answer is **YES**, and `scored` is still `false`, that means the bird just passed the pipe. We add 1 to the score and set `scored` to `true` so we don't count it again.

6. User Interface

The game has five different screens/states:

1. **Name Input Screen:** Player enters their name at startup
2. **Menu Screen:** Shows game title and player name
3. **Playing Screen:** Active gameplay with score display
4. **Paused Screen:** Overlay with "PAUSED" text
5. **Game Over Screen:** Shows final score and high score

Visual Elements:

- **Background:** Black with twinkling stars (100 stars)
- **Bird:** Gold gradient circle with eye and orange beak
- **Pipes:** 3D green gradient with highlights
- **Ground:** Brown gradient at bottom

7. Features

Feature	Description
Player Name Input	Personalized experience
Pause/Resume	Press P key or click pause button
Background Music	Looping music with toggle

Sound Effects	Game over sound
Score Tracking	Current and high score
Starry Background	100 twinkling stars
3D Styled Pipes	Gradient effects

Controls:

Key/Action	Function
SPACE / Click	Jump / Start / Restart
P	Pause / Resume
S	Open settings menu
ESC	Close settings menu
ENTER	Confirm name
BACKSPACE	Delete character in name

8. Team Work

This project was not easy. It took us three full months to complete. We started with very basic knowledge and learned a lot along the way. Here is the story of how we built it step by step.

Month 1: Struggling with Physics

"In the beginning, our bird didn't fly it teleported! We sat together for hours trying to figure out why."

The first month was the hardest. We tried to make the bird move up and down, but it looked like a robot. It would go up instantly and stop instantly. It didn't feel real.

The Fix: We learned about a concept called "Gravity." In our code, we created a number called `GRAVITY` (0.5). Every single frame (60 times a second), we added this number to the bird's speed. This made the bird fall faster and faster, just like a ball drops in real life. When we pressed the jump button, we simply set the speed to go upwards. This finally made the movement look smooth.

Month 2: The Collision Problem

Once the bird could fly, we had a new problem: crashing. We created pipes for the bird to fly through, but the game was "unfair." Sometimes the bird would crash when it wasn't even touching the pipe!

The Fix: We realized our math was too simple. We were checking if the *center* of the bird hit the pipe. But the bird has a size! We changed our code to check the *edges* of the bird (top, bottom, left, right). We basically drew an invisible box around the bird and checked if that box touched the pipe's box. This is called "AABB Collision," and it worked perfectly.

Month 3: Making it a "Real" Game

In the final month, we had a game that worked, but it was ugly and the code was messy. All our code was in one huge file, which was very confusing.

The Fix: We decided to clean up. We split our code into different files (Classes). We made a separate file for the Bird, a separate file for the Pipes, and a separate file for Sound. This made it much easier for us to work together because we weren't overwriting each other's code. We also added the "Game Over" screen, the score counter, and the background music to make it feel professional.

8.1. Team Responsibilities

This was a team effort. We made sure that **Aizaz**, **Basit**, and **Abdullah** all had an equal amount of work to do. We helped each other, but we each had our own main jobs.

Team Member	Main Tasks & Contributions
Aizaz Noor	Role: The Physics logic • Wrote the code that makes the bird fall (Gravity). • Wrote the code for the "Jump" action. • Created the "Game Loop" which makes the game run smoothly at 60 frames per second.
Abul Basit Shahid	Role: The Environment Logic • Wrote the code to create the Pipes. • Made sure the gap between pipes appears at random heights so the game is not boring. • Wrote the "Collision Detection" logic to check if the bird crashed.

Muhammad Abdullah	Role: The Player Experience • Designed all the screens (Start Menu, Settings, Game Over). • Made the keyboard (Spacebar) and Mouse controls work. • Added the Sound Manager so we can hear music and crash sounds.
--------------------------	---

9. Conclusion

This project successfully demonstrates:

- Object-Oriented Programming through modular class design
- Game physics implementation (gravity, jumping, collision)
- JavaFX application development
- Event-driven programming for user input
- Real-time graphics rendering
- Multimedia integration (audio)
- The modular design ensures that the code is maintainable, readable, and extensible.

This project was a great learning experience for Aizaz, Basit, and Abdullah. We didn't just learn how to write Java code; we learned how to solve difficult problems as a team. We are proud of our "Flappy Bird" clone because it is smooth, fun to play, and fully glitch-free.
